DevOps Pipeline Setup

GitHub Actions, CodeBuild, ECR and ECS deployment readiness

Flipkart Clone Microservices Workspace

DevOps Pipeline Setup for Flipkart Clone

This document cross-checks the DevOps pipeline readiness of the workspace and explains the recommended CI/CD setup.

PART 1: Pipeline Readiness Status

The workspace contains the files needed for AWS DevOps deployment.

Item	Status	File/Location
Dockerfile for every service	Ready	Each service folder
AWS-specific Dockerfile with DocumentDB TLS bundle	Ready	`user-service/Dockerfile.aws`, `product-service/Dockerfile.aws`, etc.
Local Docker Compose	Ready	`docker-compose.yml`
ECR push script	Ready	`push-to-ecr.sh`
ECR repository creation script	Ready	`scripts/aws-create-ecr-repositories.sh`
ECR build/push template	Ready	`scripts/aws-ecr-build-push-template.sh`
ECS force deploy script	Ready	`scripts/aws-ecs-force-deploy-template.sh`
GitHub Actions deploy workflow	Ready	`.github/workflows/deploy-to-aws-ecs.yml`
CodeBuild buildspec per service	Ready	`*/buildspec.yml`
Terraform starter	Ready	`infrastructure/terraform`
VS Code AWS linking guide	Ready	`docs/VSCODE_AWS_INFRA_AND_BUILD_GUIDE.pdf`
Monitoring guide	Ready	`docs/MONITORING_AND_OPERATIONS_GUIDE.pdf`
AWS console deployment PDF	Ready	`docs/AWS_CONSOLE_ONLY_SETUP_GUIDE.pdf`

PART 2: Best Recommended DevOps Approach

For this project, use this order:

1. Run locally using Docker Compose.
2. Push code to GitHub.
3. Create AWS infrastructure once using AWS Console or Terraform.
4. Push first Docker images to ECR using `push-to-ecr.sh`.
5. Create ECS task definitions and services.
6. Enable GitHub Actions deployment workflow for future changes.
7. Monitor with CloudWatch.

This is better than doing everything manually every time.

PART 3: GitHub Actions Deployment

Manual all-service/selected-service workflow:

```
.github/workflows/deploy-to-aws-ecs.yml
```

Automatic path-based per-service workflows:

```
.github/workflows/deploy-api-gateway.yml
.github/workflows/deploy-user-service.yml
.github/workflows/deploy-product-service.yml
.github/workflows/deploy-cart-service.yml
.github/workflows/deploy-order-service.yml
.github/workflows/deploy-payment-service.yml
.github/workflows/deploy-notification-service.yml
.github/workflows/deploy-frontend.yml
```

They can:

- Build Docker images.
- Use `Dockerfile.aws` when available.
- Push images to ECR with `latest` and Git SHA tags.
- Force ECS service deployment.
- Wait for ECS service stability.
- Automatically deploy only the service whose folder changed.

Required GitHub Secrets

Add these in GitHub repository:

```
Settings → Secrets and variables → Actions
```

Secrets:

```
AWS_ACCESS_KEY_ID
AWS_SECRET_ACCESS_KEY
AWS_REGION=us-east-1
ECS_CLUSTER=flipkart-cluster
REACT_APP_API_URL=/api
```

Optional but recommended:

```
AWS_ACCOUNT_ID
```

Manual Deployment

Go to GitHub:

```
Actions → Deploy Flipkart Clone to AWS ECS → Run workflow
```

Choose:

```
all
api-gateway
user-service
product-service
cart-service
order-service
```

```
payment-service
notification-service
frontend
```

Automatic Deployment

Push to `main` branch. The per-service workflow triggers only when that service folder changes.

Example:

```
Change api-gateway/** → deploy-api-gateway.yml runs
Change product-service/** → deploy-product-service.yml runs
Change frontend/** → deploy-frontend.yml runs
```

PART 4: AWS CodePipeline and CodeBuild Deployment

Every service has a `buildspec.yml`.

Examples:

```
api-gateway/buildspec.yml
user-service/buildspec.yml
product-service/buildspec.yml
cart-service/buildspec.yml
order-service/buildspec.yml
payment-service/buildspec.yml
notification-service/buildspec.yml
frontend/buildspec.yml
```

Each buildspec:

- Logs in to ECR.
- Builds Docker image.
- Uses `Dockerfile.aws` if present.
- Pushes `latest` and commit hash tag.
- Creates `imagedefinitions.json` for ECS deploy stage.

Required CodeBuild Environment Variables

```
AWS_ACCOUNT_ID=123456789012
AWS_DEFAULT_REGION=us-east-1
```

For frontend:

```
REACT_APP_API_URL=/api
```

Required CodeBuild Permissions

Attach to CodeBuild service role:

```
AmazonEC2ContainerRegistryPowerUser
```

Also allow ECS deploy:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "ecs:DescribeTaskDefinition",
        "ecs:RegisterTaskDefinition",
        "ecs:UpdateService",
        "ecs:DescribeServices"
      ],
      "Resource": "*"
    },
    {
      "Effect": "Allow",
      "Action": ["iam:PassRole"],
      "Resource": "arn:aws:iam::*:role/ecsTaskExecutionRole"
    }
  ]
}
```

PART 5: First Manual Push to ECR

If you want to push from VS Code terminal:

```
cd /home/user/flipkart-clone
export AWS_ACCOUNT_ID=123456789012
export AWS_REGION=us-east-1
./push-to-ecr.sh
```

Or use the env-file script:

```
cp scripts/aws-env.example scripts/aws-env.local
# edit account ID and registry
./scripts/aws-ecr-build-push-template.sh
```

PART 6: First ECS Deployment

After AWS Console ECS services are created, force deployment:

```
cp scripts/aws-env.example scripts/aws-env.local
# edit scripts/aws-env.local
./scripts/aws-ecs-force-deploy-template.sh
```

This tells ECS to pull the new `latest` images from ECR.

PART 7: DocumentDB Production Readiness

Amazon DocumentDB usually requires TLS. The AWS-specific Dockerfiles include:

```
/app/global-bundle.pem
```

Use this MongoDB URI format in ECS:

```
mongodb://docdbadmin:URL_ENCODED_PASSWORD@CLUSTER_ENDPOINT:27017/flipkart-
products?tls=true&tlsCAFile=/app/global-
bundle.pem&replicaSet=rs0&readPreference=secondaryPreferred&retryWrites=false
```

Change database name per service.

Important:

- URL encode special characters in password.
- Use `retryWrites=false`.
- Use `Dockerfile.aws` for AWS image builds.

PART 8: Service Discovery and Linking

Use Cloud Map private DNS namespace:

```
flipkart.local
```

Use these environment variables in ECS task definitions:

```
USER_SERVICE_URL=http://user-service.flipkart.local:3001
PRODUCT_SERVICE_URL=http://product-service.flipkart.local:3002
CART_SERVICE_URL=http://cart-service.flipkart.local:3003
ORDER_SERVICE_URL=http://order-service.flipkart.local:3004
PAYMENT_SERVICE_URL=http://payment-service.flipkart.local:3005
NOTIFICATION_SERVICE_URL=http://notification-service.flipkart.local:3006
```

Required security group rule:

```
flipkart-services-sg must allow inbound service-to-service traffic from itself.
```

This is needed because:

- Cart service calls product service.
- Order service calls cart and notification services.
- Payment service calls order and notification services.

PART 9: ALB Linking

ALB listener rules:

```
/api/*  → flipkart-api-tg      → api-gateway-service:3000
Default → flipkart-frontend-tg → frontend-service:80
```

Frontend should use:

```
REACT_APP_API_URL=/api
```

This makes browser requests go to:

```
http://ALB_DNS_NAME/api/...
```

Then ALB routes `/api/*` to API Gateway.

PART 10: Final Cross-Check Before Download

Run:

```
cd /home/user/flipkart-clone
./scripts/check-backend-syntax.sh
bash -n push-to-ecr.sh
bash -n scripts/aws-create-ecr-repositories.sh
bash -n scripts/aws-ecr-build-push-template.sh
bash -n scripts/aws-ecs-force-deploy-template.sh
```

Optional frontend check:

```
cd frontend
npm install
npm run build
```

If these pass, the workspace is ready to download.